

Launch Features (PRODUCT)

Gestión de Personal (CRUD de Empleados)

Permite al administrador gestionar la información esencial de los trabajadores de campo/proyecto. Incluye la creación, visualización, edición y eliminación lógica de perfiles de empleados, sentando la base para la asignación de tareas y el cálculo de pagos.

- * Listado de empleados activos con búsqueda y filtrado por nombre/rol.
- * Visualización de perfil completo del empleado.
- * Creación de nuevos empleados con datos básicos y tipo de tarifa (hora/salario).
- * Edición de información de empleados existentes.
- * Eliminación lógica de empleados.
- * Validación de datos de entrada.

Tech Involved

- * React Native (Frontend)
- * Node.js/Express (Backend)
- * PostgreSQL (Database)
- * API RESTful
- * JWT Authentication
- * ORM/Query Builder (e.g., Sequelize)

Main Requirements

- * Backend endpoints para `GET`, `POST`, `PUT`, `DELETE` empleados.
- * Validación de datos en el backend para asegurar la integridad (nombre, rol, tarifas, etc.).
- * Implementación de soft delete (`deleted\_at` timestamp).
- * Lógica de autorización basada en roles (solo admin/manager pueden modificar).
- * Pantallas móviles para listar, ver detalle y formulario de empleado.
- * Manejo de estado local y global en el frontend para datos de empleados.
- * Implementación de paginación, búsqueda y filtrado en el listado.

Registro Diario de Trabajo con Imágenes

Permite a los trabajadores registrar sus actividades diarias, incluyendo horas trabajadas, descripción y la subida de imágenes como evidencia visual (antes/después). Los administradores pueden registrar en nombre de otros y revisar todas las entradas.

- * Formulario para registrar fecha, descripción, horas trabajadas y ubicación (opcional).
- * Opción para adjuntar una o múltiples imágenes a cada entrada.
- * Previsualización de imágenes antes de subir y al ver detalles.
- * Historial de entradas de trabajo (filtrado por empleado y fecha para admin/manager).
- * Edición de entradas existentes y gestión de imágenes asociadas (añadir/eliminar).
- * Eliminación lógica de entradas de trabajo y sus imágenes.
- * Captura automática de geo-referencia y marca de tiempo en las fotos.

Tech Involved

- * React Native (Frontend)
- * Node.js/Express (Backend)
- * PostgreSQL (Database)

- * Object Storage Service (e.g., AWS S3 compatible)
- * API RESTful (Multipart/form-data)
- * Image Upload/Processing Libraries (Frontend/Backend)
- * Geolocation API (Mobile)

Main Requirements

- * Backend endpoints para `POST`, `GET`, `PUT`, `DELETE` entradas de trabajo y gestión de imágenes (`POST /{id}/images`, `DELETE /images/{id}`).
- * Manejo de subida de archivos (imágenes) de forma segura y eficiente al Object Storage.
- * Almacenamiento de URLs de imágenes en la base de datos (`work\_entry\_images` table).
- * Implementación de soft delete en cascada para entradas de trabajo e imágenes (`ON DELETE CASCADE` en FK).
- * Validación de datos de entrada de texto y archivos (tipo, tamaño).
- * Lógica de autorización: empleado solo sus entradas, admin/manager cualquiera.
- * Componentes móviles para formulario de entrada, selector de imágenes, galería y visor.
- * Visualización clara de fotos con geo-referencia y marca de tiempo.

Cálculo de Pago Semanal

Automatiza el cálculo del monto a pagar a cada empleado para un período semanal definido, basándose en las horas registradas (para empleados por hora) o salario fijo (para asalariados), considerando entradas de trabajo válidas.

- * Definición y gestión de períodos de pago semanales (fechas de inicio/fin, estado).
- * Gatillar el proceso de cálculo para un período de pago específico (solo admin).
- * Sumar horas de trabajo válidas por empleado para el período (para hourly).
- * Aplicar tarifa por hora o salario semanal fijo.
- * Almacenar resultados del cálculo por empleado y período.
- * Visualización de pagos calculados por período de pago (admin/manager).
- * Visualización del detalle de un pago calculado.
- * Capacidad de realizar ajustes manuales al monto calculado y añadir notas (solo admin).
- * Marcado de estados para períodos de pago (draft, calculated, approved).

Tech Involved

- * Node.js/Express (Backend)
- * PostgreSQL (Database)
- * Scheduled Task Mechanism (Optional for auto-calculation)
- * API RESTful

Main Requirements

- * Tablas de base de datos para `pay\_periods` y `weekly\_payments`.
- * Backend service/controller para la lógica de cálculo, consultando `employees` y `work\_entries`.
- * Endpoint para gatillar el cálculo (`POST /pay-periods/{id}/calculate`).
- * Endpoints para `GET` períodos de pago y `GET`/`PUT` pagos semanales.
- * Implementación de lógica de cálculo diferenciada por tipo de tarifa.
- * Validación: solo calcular períodos en estado 'draft', solo ajustar si el estado lo permite.
- * Lógica de autorización: admin gatilla/ajusta, admin/manager ven todos, empleado ve solo los suyos.

- * No se implementa eliminación para mantener historial.
- * Pantallas web/móviles (principalmente web para admin) para listar períodos, listar pagos por período y ver/ajustar detalle de pago.

Panel de Control del Empleador (Web)

Proporciona al cliente una interfaz centralizada para supervisar la operación de campo, gestionar personal y nómina, y acceder a informes y KPIs clave.

- * Dashboard visual con resumen del estado diario/semanal (empleados fichados, progreso visual de tareas clave).
- * Acceso a la gestión completa de Personal (CRUD - Feature 1).
- * Acceso a la revisión y filtrado de todos los Registros de Trabajo (Feature 2).
- * Acceso a la gestión y cálculo de Pago Semanal (Feature 3).
- * Visualización de Informes Profesionales Personalizados con KPIs clave.
- * Interfaz para la gestión avanzada de Tareas (asignación, seguimiento, previsión).
- * Flujo integrado para la gestión y solicitud de Materiales.
- * Interfaz configurable (casi "No-Code") para el Motor de Cálculo de Pago (variables, reglas).

Tech Involved

- * Frontend Web Framework (e.g., React, Vue, Angular)
- * Node.js/Express (Backend API, shared with mobile)
- * PostgreSQL (Database, shared)
- * Object Storage Service (Image display, shared)
- * Charting/Data Visualization Library
- * Reporting Engine/Library (Backend/Frontend)

Main Requirements

- * Desarrollo de una aplicación web con autenticación para el rol de empleador.
- * Implementación de UI/UX adaptada a pantalla grande para gestión de tablas y dashboards.
- * Integración con los mismos endpoints backend utilizados por la app móvil para Personal, Trabajo y Pagos.
- * Desarrollo de módulos frontend y backend para la gestión de Tareas y Materiales.
- * Diseño e implementación de la lógica backend para calcular y generar los KPIs personalizados.
- * Desarrollo frontend para visualizar los KPIs en dashboards y generar reportes (exportables, ej. PDF, CSV).
- * Diseño e implementación de la interfaz "No-Code" y la lógica backend para configurar el motor de cálculo de pago.

Future Features

Gestión Avanzada de Tareas

- * Creación, asignación y seguimiento de tareas a empleados/equipos.
- * Definición de subtareas y dependencias.
- * Estimación de tiempo y recursos (materiales) por tarea.
- * Visualización tipo Gantt o calendario de tareas asignadas.

- * Alertas de solapamiento de tareas o conflictos de materiales durante la asignación.

Tech Involved

- * PostgreSQL (Database, additional tables for tasks)
- * Backend Services/APIs for Task Management
- * Frontend Web Components (Calendar/Gantt views)

Main Requirements

- * Diseño de esquema de base de datos para modelar tareas, asignaciones, dependencias.
- * Implementación de lógica backend para la previsión de solapamientos y conflictos.
- * Desarrollo de endpoints API para CRUD de tareas y asignaciones.
- * Desarrollo de interfaces web para la gestión visual de tareas.

Gestión de Materiales

- * Catálogo de materiales.
- * Registro de inventario de materiales.
- * Solicitud de materiales por parte de empleados de campo a través de la app móvil.
- * Aprobación/Rechazo de solicitudes por parte del administrador/manager (web).
- * Asociación de materiales requeridos a tareas específicas.

Tech Involved

- * PostgreSQL (Database, new tables for materials, inventory, requests)
- * Backend Services/APIs for Material Management
- * Mobile App Forms (Material Requests)
- * Frontend Web Interfaces (Inventory, Approvals)

Main Requirements

- * Diseño de esquema de base de datos para materiales e inventario.
- * Desarrollo de endpoints API para gestionar materiales, inventario y solicitudes.
- * Implementación de flujo de solicitud y aprobación.
- * Integración en la app móvil para empleados y en la web para administradores.

Módulo de Reportes Personalizados Avanzado

- * Generador de reportes drag-and-drop o basado en plantillas (web).
- * Exportación de reportes en múltiples formatos (PDF, CSV, Excel).
- * Programación de generación automática de reportes.
- * Dashboards interactivos con filtros dinámicos.
- * Análisis de tendencias y comparativas históricas.

Tech Involved

- * Reporting Engine/Library (Backend/Frontend)
- * Data Warehousing concepts (potentially for complex reporting on large data)
- * Advanced Charting/Data Visualization Libraries

Main Requirements

- * Selección e integración de una herramienta/librería de reporting flexible.
- * Desarrollo de interfaz para definir y configurar reportes.
- * Implementación de lógica backend para consultar datos complejos y generar reportes.

- * Gestión de la programación y entrega de reportes.

Módulo de Fichaje / Reloj Digital con Validación Fotográfica

- * Registro de entrada y salida de jornada laboral por parte del empleado a través de la app móvil.
- * Requerimiento de una foto del empleado en el momento del fichaje.
- * Captura automática de geo-referencia y marca de tiempo en la foto de fichaje.
- * Validación de la ubicación del fichaje (dentro de áreas de proyecto permitidas).
- * Visualización en tiempo real (o casi) del estado de fichaje de los empleados (web para admin).
- * Reportes de asistencia y puntualidad.

Tech Involved

- * PostgreSQL (Database, new table for clock-in/out)
- * Backend Services/APIs for Clock-in/out
- * Mobile App UI for Clock-in/out
- * Image Upload (reusing Feature 2 tech)
- * Geolocation API (Mobile)
- * Geofencing (optional, for location validation)

Main Requirements

- * Diseño de esquema de base de datos para registros de fichaje.
- * Desarrollo de endpoints API para registrar fichajes con foto y ubicación.
- * Integración en la app móvil de la funcionalidad de fichaje.
- * Lógica backend para validar la foto (si es posible con IA básica para detectar rostro) y la ubicación.
- * Desarrollo de interfaz web para visualizar el estado de fichaje y generar reportes de asistencia.

Módulo de Anticipos y Descuentos

- * Registro de anticipos entregados a empleados.
- * Configuración de tipos de descuentos (ej. multas, uniformes).
- * Asociación de anticipos/descuentos a empleados.
- * Integración de anticipos/descuentos en el cálculo final del pago semanal (Feature 3).
- * Reportes de anticipos y descuentos por empleado.

Tech Involved

- * PostgreSQL (Database, new tables for advances, deductions, employee balances)
- * Backend Services/APIs
- * Frontend Web Interfaces

Main Requirements

- * Diseño de esquema de base de datos para gestionar anticipos y descuentos.
- * Modificación de la lógica del Motor de Cálculo de Pago (Feature 3) para incluir anticipos y descuentos.
- * Desarrollo de endpoints API y interfaces web para registrar y gestionar anticipos/descuentos.

System Diagram

```
``svg
<svg width="800" height="600"
xmlns="http://www.w3.org/2000/svg"(http://www.w3.org/2000/svg)">
<style>
  .shape {
    stroke="#000";
    stroke-width="2";
    rx="10";
    ry="10";
  }
  .label {
    font-family="Arial, sans-serif";
    font-size="14px";
    fill="#000";
    text-anchor="middle";
  }
  .arrow {
    stroke="#000";
    stroke-width="2";
    marker-end="url(#arrowhead)";
    fill="none";
  }
  .color-frontend { fill="#a7f3d0"; } /* Green-ish */
  .color-backend { fill="#bfd5bf"; } /* Blue-ish */
  .color-database { fill="#fecaca"; } /* Red-ish */
  .color-storage { fill="#fed7aa"; } /* Orange-ish */
  .color-external { fill="#e9d5ff"; } /* Purple-ish */
  .color-employer-web { fill="#bae6fd"; } /* Light Blue */
</style>
<defs>
  <marker id="arrowhead" markerWidth="10" markerHeight="7" refX="10" refY="3.5"
orient="auto">
    <polygon points="0 0, 10 3.5, 0 7" />
  </marker>
</defs>

<rect x="50" y="100" width="150" height="80" class="shape color-frontend"></rect>
<text x="125" y="145" class="label">Mobile App</text>
<text x="125" y="165" class="label">(Empleados)</text>

<rect x="50" y="250" width="150" height="80" class="shape color-employer-web"></rect>
<text x="125" y="295" class="label">Web App</text>
<text x="125" y="315" class="label">(Empleador)</text>

<rect x="300" y="180" width="200" height="80" class="shape color-backend"></rect>
<text x="400" y="225" class="label">Backend API</text>
```

```

<text x="400" y="245" class="label">(Node.js/Express)</text>

<rect x="550" y="100" width="150" height="80" class="shape color-database"></rect>
<text x="625" y="145" class="label">Database</text>
<text x="625" y="165" class="label">(PostgreSQL)</text>

<rect x="550" y="250" width="150" height="80" class="shape color-storage"></rect>
<text x="625" y="295" class="label">Image Storage</text>
<text x="625" y="315" class="label">(S3 Compatible)</text>

<rect x="375" y="400" width="150" height="80" class="shape color-external"></rect>
<text x="450" y="445" class="label">External Services</text>
<text x="450" y="465" class="label">(Payments, Analytics)</text>

<path d="M200 140 H 300" class="arrow"></path>
<text x="250" y="135" class="label" style="font-size:10px;">API Calls</text>

<path d="M200 290 H 300" class="arrow"></path>
<text x="250" y="285" class="label" style="font-size:10px;">API Calls</text>

<path d="M500 220 H 550" class="arrow"></path>
<text x="525" y="215" class="label" style="font-size:10px;">Read/Write</text>

<path d="M500 230 C 520 270, 530 290, 550 290" class="arrow"></path>
<text x="525" y="265" class="label" style="font-size:10px;
text-anchor:start;">Upload</text>
<text x="525" y="278" class="label" style="font-size:10px;
text-anchor:start;">Manage</text>

<path d="M450 260 V 400" class="arrow"></path>
<text x="455" y="330" class="label" style="font-size:10px;">Integrations</text>

<path d="M550 290 H 200 V 160" class="arrow" marker-end="url(#arrowhead)"></path>
<text x="375" y="305" class="label" style="font-size:10px;">Image Display</text>
<path d="M550 290 H 200 V 300" class="arrow" marker-end="url(#arrowhead)"></path>
<text x="375" y="275" class="label" style="font-size:10px;">Image Display</text>

</svg>

```

Questions & Clarifications

- **Roles y Permisos:** ¿Existen otros roles además de 'employee' y 'admin' (ej. 'manager' con permisos intermedios)? La descripción menciona 'manager' con algunos permisos, pero la matriz detallada solo cubre 'employee' y 'admin'. ¿Cuáles serían los permisos exactos para 'manager'?
 - Sí, aquí en la piscifactoría necesitamos un rol intermedio. Piense en nuestros *capataces*. Ellos no son dueños como yo (el 'admin'), pero están a cargo de grupos de trabajadores en diferentes áreas o turnos. Necesitan ver qué está haciendo su gente, sus registros de trabajo, quizás aprobar solicitudes de materiales para sus equipos. Pero no deben poder cambiar la información de pago ni eliminar empleados de la base de datos. El 'manager' sería el capataz, que ve casi todo de sus asignados y reporta a mí.
- **Motor de Cálculo de Pago:** La "Creative Direction" menciona un motor "No-Code" configurable. ¿Qué nivel de flexibilidad se espera para esta configuración? ¿Se limitará a tarifas y descuentos básicos, o se deben contemplar reglas más complejas (ej. pago por tarea, pago extra por tipo de tarea/ubicación, reglas de horas extra, etc.)?
 - Necesitamos algo flexible, pero sin volvernos locos. Lo básico es hora o salario fijo, eso está claro. Pero a veces pagamos un extra por tareas específicas, digamos, si hay que trabajar metido en el agua fría revisando redes, o por sacar una cosecha de peces en un día caluroso. También manejamos anticipos pequeños y descuentos por uniformes o herramientas que pierden. Que sea configurable para añadir estos "extras" o "descuentos" simples por tipo de tarea o una bonificación fija por periodo sería ideal. No necesitamos cálculo de impuestos complejos ni nada de eso, solo lo que afecta al pago bruto semanal aquí en la granja.
- **Validación de Ubicación (Fichaje y Entradas de Trabajo):** Si bien se menciona la captura de geo-referencia, ¿se requiere alguna validación estricta de que el fichaje o la entrada de trabajo se realizó dentro de una zona geográfica permitida (Geofencing)? ¿Quién definirá estas zonas?
 - Sí, es importante para nosotros. Nuestros estanques están dispersos. Necesitamos saber que cuando alguien ficha o registra que trabajó en el estanque 5, realmente estuvo cerca del estanque 5. No tan estricto como un banco, pero sí una validación para que no fichen desde sus casas. Yo definiría las zonas, quizás marcando en un mapa sencillo dónde están los límites de la granja o áreas de trabajo principales.
- **Integración con Sistemas de Nómina/Contabilidad:** ¿Existe alguna necesidad a futuro de integrar los resultados del cálculo de pago con un sistema de nómina o contabilidad externo?
 - Por ahora, no. La idea es que esta aplicación sea la única fuente para saber cuánto hay que pagarle a cada trabajador de campo cada semana. Los resultados calculados aquí son lo que usamos para hacer las transferencias o dar el efectivo. Si en el futuro crecemos mucho o la ley lo exige, podríamos pensarlo, pero de momento, no es una prioridad ni un requisito.
- **Generación de Períodos de Pago:** ¿Los períodos de pago se crearán manualmente por el administrador a través de la web, o se generarán automáticamente por el sistema (ej. cada semana)?
 - Lo ideal sería que se generen automáticamente. Que yo configure que nuestro período es de lunes a domingo, y el sistema cree el próximo período

automáticamente cuando termina el anterior, quizás con una o dos semanas de anticipación para que yo pueda ir viendo. Si necesito uno especial, una opción manual para crearlo estaría bien, pero lo usual debe ser automático.

- **Manejo de Offline en la App Móvil:** Dado que los trabajadores de campo podrían estar en áreas con conectividad limitada, ¿es un requisito que la app móvil funcione offline para registrar entradas de trabajo y fichajes, y sincronice los datos una vez que haya conexión? Esto impactaría significativamente la arquitectura y complejidad del frontend y backend.
 - ¡Absolutamente crucial! Hay zonas por los estanques donde la señal es mala o nula. Los muchachos *tienen* que poder fichar y registrar su trabajo y tomar fotos aunque no tengan internet en ese momento. La aplicación debe guardar todo localmente y que, cuando vuelvan a tener señal, automáticamente se suba la información. Si no, esto no nos sirve.
- **Tipos de Archivos de Imagen:** ¿Se admitirán solo los tipos de imagen comunes (JPG, PNG) o se requieren otros formatos (ej. HEIC de iPhone)? ¿Hay un tamaño máximo por archivo o por entrada de trabajo?
 - Principalmente fotos de celulares, así que JPG y HEIC son los más comunes. PNG si hacen una captura de pantalla, quizás. Que soporte esos sería suficiente. En cuanto al tamaño, no queremos que suban archivos gigantes que nos llenen el almacenamiento, pero tampoco tan pequeños que no se vea bien. Unos 5 MB por foto me parece razonable, y quizás un límite de 3-5 fotos por entrada de trabajo para mantenerlo manejable. Lo importante es que se vea claro el trabajo hecho.
- **Flujo de Materiales:** Para la gestión de materiales, ¿se requiere un sistema de inventario completo con seguimiento de entrada/salida, o solo la solicitud y aprobación de materiales?
 - Por ahora, con la solicitud y aprobación me basta. Que el trabajador pueda decir "Necesito 2 bolsas de alimento y una red para el estanque 3" y que el capataz o yo lo veamos en la web y podamos aprobar o rechazar. Un inventario completo sería genial a futuro, para saber cuánto nos queda de cada cosa, pero no es una necesidad inicial. La prioridad es controlar qué pide la gente y para dónde.

List of Architecture Consideration Questions

- ¿Cómo se manejará el versionado de la API para futuras actualizaciones sin romper la compatibilidad con versiones anteriores de la aplicación móvil?
 - Desde la perspectiva de un Lead Coder con enfoque en SOLID, manejar el versionado de API es clave para la mantenibilidad y el Principio Abierto/Cerrado (OCP). Deberíamos diseñar la API con versiones explícitas en la URL (ej. `/api/v1/employees`). Las nuevas funcionalidades o cambios rupturistas se introducen en nuevas versiones (ej. `/api/v2/employees`). Esto permite que las aplicaciones móviles más antiguas sigan funcionando con `v1` mientras se desarrolla la migración a `v2`. Internamente en el backend, podemos usar el Principio de Inversión de Dependencias (DIP) y Segregación de Interfaces (ISP) para que los controladores de diferentes versiones de la API dependan de abstracciones o

interfaces de servicios de negocio comunes, minimizando la duplicación de lógica y facilitando la extensión (OCP) con nuevas versiones de API sin modificar drásticamente los servicios existentes.

- ¿Qué estrategia de escalabilidad se implementará para el backend y la base de datos a medida que aumente el número de empleados y entradas de trabajo? (Horizontal scaling, read replicas, sharding).
 - La escalabilidad es vital. Para el backend, la arquitectura basada en Node.js/Express es inherentemente adecuada para la escalabilidad horizontal (añadir más instancias). Aquí aplicamos el Principio de Responsabilidad Única (SRP) al mantener los servicios de backend sin estado (stateless), lo que facilita clonar instancias detrás de un balanceador de carga. Para la base de datos (PostgreSQL), la primera línea de defensa es optimizar consultas con índices adecuados. A medida que crezca el volumen de datos, consideraremos réplicas de lectura para distribuir la carga de consultas (**GET**) y, eventualmente, estrategias de sharding si la escritura se vuelve un cuello de botella, aunque esto último añade complejidad y debería basarse en métricas reales de crecimiento. La elección de PostgreSQL, al ser robusta, nos da un buen margen antes de necesitar soluciones de sharding.
- ¿Cómo se garantizará la seguridad de los datos sensibles, especialmente la información de pago y los datos personales de los empleados? (Encryption at rest, in transit, access controls).
 - La seguridad debe ser una capa transversal. Implementaremos cifrado de datos en tránsito usando HTTPS/SSL para todas las comunicaciones entre las aplicaciones frontend y el backend. En la base de datos, los datos sensibles (como tarifas de pago) se cifrarán en reposo. El control de acceso se basará estrictamente en la autenticación (JWT) y autorización por roles, validando en cada endpoint de backend si el usuario tiene permiso para acceder o modificar esos datos, adhiriéndonos al Principio de Mínimo Privilegio. En el código, aplicaremos SRP para segregar la lógica de seguridad (autenticación, autorización) en módulos dedicados, y DIP para depender de abstracciones de servicios de seguridad que puedan ser implementadas por diferentes proveedores o lógicas sin afectar el código de negocio principal.
- ¿Cuál será la estrategia de respaldo y recuperación de datos para la base de datos y el almacenamiento de objetos?
 - Implementaremos copias de seguridad automáticas y regulares tanto para la base de datos PostgreSQL como para el servicio de almacenamiento de objetos. Para la base de datos, se configurarán Point-in-Time Recovery (PITR) si el proveedor de hosting gestionado lo permite, para poder restaurar a un momento específico en caso de corrupción de datos o error humano. Los backups se almacenarán en una ubicación geográfica diferente a la de los datos primarios. Para el almacenamiento de objetos, se configurará replicación entre regiones si es crítico, o al menos copias de seguridad periódicas a otro bucket/ubicación. Tendremos un plan de recuperación documentado y probado periódicamente para asegurar que podemos

restaurar el servicio y los datos en un tiempo aceptable después de un incidente.

- ¿Cómo se gestionará la autenticación y autorización entre la aplicación móvil, la aplicación web y el backend API? (JWT, OAuth2, etc.)
 - Utilizaremos JWT (JSON Web Tokens) para la autenticación. El usuario inicia sesión, el backend verifica sus credenciales y emite un JWT que el cliente almacena de forma segura. Este token se envía en el encabezado **Authorization** de cada solicitud protegida. En el backend, un middleware validará el token en cada request. La autorización se basará en los roles incrustados en el token (o consultados desde la DB si los roles son muy dinámicos). Implementaremos la lógica de verificación de permisos en puntos de entrada de la API (controladores), utilizando principios como SRP para aislar esta lógica de autorización y DIP para poder cambiar la implementación de la validación de roles si fuera necesario. Esto garantiza que las reglas de acceso se apliquen consistentemente en todos los puntos de entrada, sin importar si la solicitud viene de la app móvil o la web.
- ¿Se implementará una arquitectura de microservicios o un monolito modular para el backend? ¿Cuáles son los trade-offs?
 - Dada la descripción inicial del cliente único y el conjunto definido de funcionalidades, una arquitectura de **monolito modular** es probablemente la más adecuada y eficiente para empezar. Un monolito modular aplica SRP a nivel de módulos/servicios dentro de una única aplicación (ej. módulos separados para Empleados, Trabajo, Pago), lo que mantiene el código organizado y cohesivo. Los trade-offs son que escalar una parte específica puede requerir escalar todo el monolito, y un error en un módulo podría afectar a otros (aunque la modularidad ayuda a contener esto). Migrar a microservicios (donde cada módulo se convierte en un servicio independiente) sería una opción a futuro si la aplicación crece significativamente en complejidad, equipo de desarrollo, o si partes específicas necesitan escalar de forma independiente. Los microservicios ofrecen mayor escalabilidad y resiliencia a fallos de módulos, pero añaden mucha complejidad operativa (despliegue, comunicación entre servicios, monitoreo distribuido). Para este proyecto, empezar modular es lo más pragmático, aplicando SRP y DIP dentro del monolito para mantener la puerta abierta a una futura extracción de servicios si es necesario (OCP).
- ¿Cómo se manejarán las tareas de procesamiento pesado o asíncrono en el backend (ej. generación de reportes complejos, procesamiento de imágenes)? (Message queues, worker processes).
 - Las tareas que pueden llevar tiempo (subir y quizás redimensionar imágenes, cálculos de pago complejos si la nómina crece, generación de reportes) no deben bloquear la respuesta de la API principal. Implementaremos un sistema de tareas asíncronas utilizando **colas de mensajes** (ej. RabbitMQ, SQS, Pub/Sub). Cuando una tarea larga necesite ejecutarse (ej. al subir una imagen, la API guarda la referencia y responde rápido; un mensaje se envía a la cola para redimensionar), un **proceso worker** separado consume mensajes de la cola y ejecuta la tarea en segundo plano. Esto adhiere al Principio de Responsabilidad Única (SRP), separando la responsabilidad de

recibir requests API de la responsabilidad de procesar tareas pesadas. También mejora la experiencia de usuario al hacer que las APIs respondan más rápido.

- ¿Cómo se realizará el monitoreo del rendimiento, errores y la salud general del sistema en producción?
 - Configuraremos herramientas de **Application Performance Monitoring (APM)** (ej. New Relic, Datadog, Prometheus + Grafana) para obtener métricas clave del backend (latencia de requests, tasa de errores, uso de recursos), la base de datos (consultas lentas, uso de conexiones) y el servicio de almacenamiento. Implementaremos **logging estructurado** centralizado (ej. ELK stack, Splunk, CloudWatch Logs) para agregar y buscar logs de errores y eventos relevantes. Se configurarán **alertas automáticas** basadas en umbrales críticos (altas tasas de error 5xx, latencia elevada persistente, falta de espacio en disco, fallos en tareas programadas). Esto nos permite detectar problemas de forma proactiva, diagnosticar rápidamente usando logs correlacionados y asegurar la salud del sistema.
- ¿Cuál será la estrategia de despliegue (CI/CD) para el backend y las aplicaciones frontend?
 - Estableceremos un pipeline de **Integración Continua/Despliegue Continuo (CI/CD)**. Cada cambio en el código pasará por:
 1. **Integración Continua (CI)**: Ejecución automática de tests (unitarios, de integración), análisis estático de código (linters, formatters), y construcción de artefactos desplegable (imágenes Docker para backend, bundles para frontend). Esto asegura que solo el código de calidad llega a las etapas posteriores.
 2. **Despliegue Continuo (CD)**: Una vez que la CI pasa, los artefactos se despliegan automáticamente en entornos de staging para pruebas adicionales (automáticas o manuales). Una aprobación (manual o automática) gatilla el despliegue a producción, usando estrategias que minimicen el tiempo de inactividad (ej. rolling updates). Esto automatiza y estandariza el proceso de liberación, reduciendo errores manuales y permitiendo entregas rápidas y confiables.
- ¿Cómo se gestionarán las configuraciones de entorno (desarrollo, staging, producción)?
 - Las configuraciones (credenciales de base de datos, claves de API externas, URLs de servicios, puertos, etc.) **nunca** se codificarán directamente en el código fuente. Se gestionarán utilizando **variables de entorno**. En desarrollo local, se pueden usar archivos `.env` (ignorados por Git). En staging y producción, el proveedor de hosting o la orquestación de contenedores (Docker/Kubernetes) se encargará de inyectar estas variables de entorno de forma segura en el runtime de las aplicaciones. Esto adhiere al principio de "Configuración en el Entorno" de los 12 Factores, desacoplando la configuración del código y facilitando el despliegue en diferentes entornos sin modificar el código fuente.